

Reference › Kubectl › Overview

Command line tool (kubectl) | Kubernetes

Command line tool (kubectl) Kubernetes provides a command line tool for communicating with a Kubernetes cluster's

control plane,

using the Kubernetes API. This tool is named kubectl. For configuration, kubectl looks for a file named config in the \$HOME/.kube directory.

You can specify other kubeconfig

files by setting the KUBECONFIG environment variable or by setting the

--kubeconfig flag. This overview covers kubectl syntax, describes the command operations, and provides common examples.

For details about each command, including all the supported flags and subcommands, see the kubectl reference documentation. For an overview, see The kubectl command-line tool.

For installation instructions, see Installing kubectl;

for a quick guide, see the cheat sheet.

If you're used to using the docker command-line tool,

kubectl for Docker Users explains some equivalent commands for Kubernetes. Syntax Use the following syntax

to run kubectl commands from your terminal window: kubectl [command] [TYPE] [NAME] [flags]

where command, TYPE, NAME, and flags are: command: Specifies the operation that you want to perform on one or more resources,

for example create, get, describe, delete. TYPE: Specifies the resource type. Resource types are case-insensitive and

you can specify the singular, plural, or abbreviated forms.

For example, the following commands produce the same output: kubectl get pod pod1

kubectl get pods pod1

kubectl get po pod1

NAME: Specifies the name of the resource. Names are case-sensitive. If the name is omitted, details for all resources are displayed, for example kubectl get pods. When performing an operation on multiple resources, you can specify each resource by

type and name or specify one or more files: To group resources if they are all the same type: TYPE1 name1 name2 name<#>. Example: kubectl get pod example-pod1

example-pod2 To specify multiple resource types individually: TYPE1/name1 TYPE1/name2 TYPE2/name3

TYPE<#>/name<#>. Example: kubectl get pod/example-pod1 replicationcontroller/example-rc1 To specify

resources with one or more files: -f file1 -f file2 -f file<#> Use YAML rather than JSON

since YAML tends to be more user-friendly, especially for configuration files. Example: kubectl get -f

./pod.yaml flags: Specifies optional flags. For example, you can use the -s or --server flags

to specify the address and port of the Kubernetes API server. Caution: Flags that you specify from the command

line override default values and any corresponding environment variables. If you need help, run kubectl help

from the terminal window. In-cluster authentication and namespace overrides By default kubectl will first determine if it is running within a pod, and thus in a cluster.

It starts by checking for the KUBERNETES_SERVICE_HOST and KUBERNETES_SERVICE_PORT environment

variables and the existence of a service account token file at

/var/run/secrets/kubernetes.io/serviceaccount/token.

If all three are found in-cluster authentication is assumed. To maintain backwards compatibility, if the

POD_NAMESPACE environment variable is set

during in-cluster authentication it will override the default namespace from the

service account token. Any manifests or tools relying on namespace defaulting will be affected by

this. POD_NAMESPACE environment variable If the POD_NAMESPACE environment variable is set, cli

operations on namespaced resources

will default to the variable value. For example, if the variable is set to seattle,

kubectl get pods would return pods in the seattle namespace. This is because pods are

a namespaced resource, and no namespace was provided in the command. Review the output of `kubectl api-resources` to determine if a resource is namespaced. Explicit use of `--namespace <value>` overrides this behavior. How `kubectl` handles ServiceAccount tokens: If there is a Kubernetes service account token file mounted at

`/var/run/secrets/kubernetes.io/serviceaccount/token`, and the `KUBERNETES_SERVICE_HOST` environment variable is set, and the `KUBERNETES_SERVICE_PORT` environment variable is set, and you don't explicitly specify a namespace on the `kubectl` command line then `kubectl` assumes it is running in your cluster. The `kubectl` tool looks up the

namespace of that ServiceAccount (this is the same as the namespace of the Pod) and acts against that namespace. This is different from what happens outside of a cluster; when `kubectl` runs outside a cluster and you don't specify a namespace, the `kubectl` command acts against the namespace set for the current context in your client configuration. To change the default namespace for your `kubectl` you can use the following command: `kubectl config set-context --current --namespace=<namespace-name>`

Operations The following table includes short descriptions and the general syntax for all of the `kubectl` operations:

Operation	Syntax	Description
<code>alpha</code>	<code>kubectl alpha SUBCOMMAND [flags]</code>	List the available commands that correspond to alpha features, which are not enabled in Kubernetes clusters by default.
<code>annotate</code>	<code>kubectl annotate (-f FILENAME TYPE NAME TYPE/NAME) KEY_1=VAL_1 ... KEY_N=VAL_N [--overwrite] [--all] [--resource-version=version] [flags]</code>	Add or update the annotations of one or more resources.
<code>api-resources</code>	<code>kubectl api-resources [flags]</code>	List the API resources that are available.
<code>api-versions</code>	<code>kubectl api-versions [flags]</code>	List the API versions that are available.
<code>apply</code>	<code>kubectl apply -f FILENAME [flags]</code>	Apply a configuration change to a resource from a file or stdin.
<code>attach</code>	<code>kubectl attach POD -c CONTAINER [-i] [-t] [flags]</code>	Attach to a running container either to view the output stream or interact with the container (stdin).
<code>auth</code>	<code>kubectl auth [flags] [options]</code>	Inspect authorization.
<code>autoscale</code>	<code>kubectl autoscale (-f FILENAME TYPE NAME TYPE/NAME) [--min=MINPODS] --max=MAXPODS [--cpu=CPU] [flags]</code>	Automatically scale the set of pods that are managed by a replication controller.
<code>certificate</code>	<code>kubectl certificate SUBCOMMAND [options]</code>	Modify certificate resources.
<code>cluster-info</code>	<code>kubectl cluster-info [flags]</code>	Display endpoint information about the master and services in the cluster.
<code>completion</code>	<code>kubectl completion SHELL [options]</code>	Output shell completion code for the specified shell (bash or zsh).
<code>config</code>	<code>kubectl config SUBCOMMAND [flags]</code>	Modifies kubeconfig files. See the individual subcommands for details.
<code>convert</code>	<code>kubectl convert -f FILENAME [options]</code>	Convert config files between different API versions. Both YAML and JSON formats are accepted. Note - requires <code>kubectl-convert</code> plugin to be installed.
<code>cordon</code>	<code>kubectl cordon NODE [options]</code>	Mark node as unschedulable.
<code>cp</code>	<code>kubectl cp <file-spec-src> <file-spec-dest> [options]</code>	Copy files and directories to and from containers.
<code>create</code>	<code>kubectl create -f FILENAME [flags]</code>	Create one or more resources from a file or stdin.
<code>delete</code>	<code>kubectl delete (-f FILENAME TYPE [NAME /NAME -l label --all]) [flags]</code>	Delete resources either from a file, stdin, or specifying label selectors, names, resource selectors, or resources.
<code>describe</code>	<code>kubectl describe (-f FILENAME TYPE [NAME_PREFIX /NAME -l label]) [flags]</code>	Display the detailed state of one or more resources.
<code>diff</code>	<code>kubectl diff -f FILENAME [flags]</code>	Diff file or stdin against live configuration.
<code>drain</code>	<code>kubectl drain NODE [options]</code>	Drain node in preparation for maintenance.
<code>edit</code>	<code>kubectl edit (-f FILENAME TYPE NAME TYPE/NAME) [flags]</code>	Edit and update the definition of one or more resources on the server by using the default editor.
<code>events</code>	<code>kubectl events</code>	List eventsexec
<code>exec</code>	<code>kubectl exec POD [-c CONTAINER] [-i] [-t] [flags] [-- COMMAND [args...]]</code>	Execute a command against a container in a pod.
<code>explain</code>	<code>kubectl explain TYPE [--recursive=false] [flags]</code>	Get documentation of various resources. For instance pods, nodes, services, etc.
<code>expose</code>	<code>kubectl expose (-f FILENAME TYPE NAME TYPE/NAME) [--port=port] [--protocol=TCP UDP] [--target-port=number-or-name] [--name=name] [--external-ip=external-ip-of-service] [--type=type] [flags]</code>	Expose a replication controller, service, or pod as a new Kubernetes service.
<code>get</code>	<code>kubectl get (-f FILENAME TYPE [NAME /NAME -l label]) [--watch] [--sort-by=FIELD] [--o --output]=OUTPUT_FORMAT [flags]</code>	List one or more resources.
<code>kustomize</code>	<code>kubectl kustomize <dir> [flags] [options]</code>	List a set of API resources generated from instructions in a kustomization.yaml file. The argument must be the path to the directory containing the file, or a git repository URL with a path suffix specifying same with respect to the repository root.
<code>label</code>	<code>kubectl label (-f FILENAME TYPE NAME TYPE/NAME) KEY_1=VAL_1 ... KEY_N=VAL_N [--overwrite] [--all] [--resource-version=version] [flags]</code>	Add or update the labels of one or more resources.
<code>logs</code>	<code>kubectl logs POD [-c CONTAINER] [--follow] [flags]</code>	Print the logs for a container in a pod.
<code>options</code>	<code>kubectl options</code>	List of global command-line options, which apply to all commands.
<code>patch</code>	<code>kubectl patch (-f FILENAME TYPE NAME TYPE/NAME) --patch PATCH [flags]</code>	Update one or more fields of a resource by using the strategic merge patch

process.plugin kubectl plugin [flags] [options] Provides utilities for interacting with plugins.

port-forward kubectl port-forward POD [LOCAL_PORT:]REMOTE_PORT [...] [LOCAL_PORT_N:]REMOTE_PORT_N [flags] Forward one or more local ports to a pod.

proxy kubectl proxy [--port=PORT] [--www=static-dir] [--www-prefix=prefix] [--api-prefix=prefix] [flags] Run a proxy to the Kubernetes API server.

replace kubectl replace -f FILENAME Replace a resource from a file or stdin.

rollout kubectl rollout SUBCOMMAND [options] Manage the rollout of a resource. Valid resource types include: deployments, daemonsets and statefulsets.

run kubectl run NAME --image=image [--env="key=value"] [--port=port] [--dry-run=server|client|none] [--overrides=inline-json] [flags] Run a specified image on the cluster.

scale kubectl scale (-f FILENAME | TYPE NAME | TYPE/NAME) --replicas=COUNT [--resource-version=version] [--current-replicas=count] [flags] Update the size of the specified replication controller.

set kubectl set SUBCOMMAND [options] Configure application resources.

taint kubectl taint NODE NAME KEY_1=VAL_1:TAINT_EFFECT_1 ... KEY_N=VAL_N:TAINT_EFFECT_N [options] Update the taints on one or more nodes.

top kubectl top (POD | NODE) [flags] [options] Display Resource (CPU/Memory/Storage) usage of pod or node.

uncordon kubectl uncordon NODE [options] Mark node as schedulable.

version kubectl version [--client] [flags] Display the Kubernetes version running on the client and server.

wait kubectl wait [--f FILENAME] | resource.group/resource.name | resource.group [--l label | --all]) [--for=delete|--for condition=available] [options] Experimental: Wait for a specific condition on one or many resources. To learn more about command operations, see the kubectl reference documentation.

Resource types The following table includes a list of all the supported resource types and their abbreviated aliases. (This output can be retrieved from kubectl api-resources, and was accurate as of Kubernetes 1.25.0)

NAME	SHORTNAME	APIVERSION	NAME SPACED
Kind	bindings	v1	true
Binding	componentstatuses	sv1	false
ComponentStatus	configmaps	cmv1	true
ConfigMap	endpoints	sepv1	true
Endpoint	events	sevv1	true
Event	limitranges	limitsv1	true
LimitRange	namespaces	ns	v1
false	Namespaces	nodes	nov1
false	Node	persistentvolumeclaims	pvcv1
true	PersistentVolumeClaim	persistentvolumes	pv
false	PersistentVolume	pod	pov1
true	Pod	podtemplates	sv1
true	PodTemplate	replicationcontrollers	rcv1
true	ReplicationController	resourcequotas	quotav1
true	ResourceQuota	secrets	sv1
true	Secret	serviceaccounts	sa
v1	ServiceAccount	services	svcv1
true	Service	mutatingwebhookconfigurations	admissionregistration.k8s.io/v1
false	MutatingWebhookConfiguration	validatingwebhookconfigurations	admissionregistration.k8s.io/v1
false	ValidatingWebhookConfiguration	customresourcedefinitions	crd, crds
apiextensions.k8s.io/v1	false	CustomResourceDefinition	apiservices
apiregistration.k8s.io/v1	false	APIService	controllerrevisions
apps/v1	true	ControllerRevision	daemonsets
ds	apps/v1	true	DaemonSet
deployments	deploy	apps/v1	true
Deployment	replicasets	rs	apps/v1
true	ReplicaSet	statefulsets	sts
apps/v1	true	StatefulSet	tokenreviews
authentication.k8s.io/v1	false	TokenReview	localsubjectaccessreviews
authorization.k8s.io/v1	true	LocalSubjectAccessReview	selfsubjectaccessreviews
authorization.k8s.io/v1	false	SelfSubjectAccessReview	selfsubjectrulesreviews
authorization.k8s.io/v1	false	SelfSubjectRulesReview	subjectaccessreviews
authorization.k8s.io/v1	false	SubjectAccessReview	horizontalpodautoscalers
shpaautoscaling/v2	true	HorizontalPodAutoscaler	cronjobs
batch/v1	true	CronJob	jobs
batch/v1	true	Job	certificatesigningrequests
cs	certificates.k8s.io/v1	false	CertificateSigningRequest
leases	coordination.k8s.io/v1	true	Lease
endpointslices	discovery.k8s.io/v1	true	EndpointSlice
events	events.k8s.io/v1	true	Event
flowschemas	flowcontrol.apiserver.k8s.io/v1	beta2	false
FlowSchema	prioritylevelconfigurations	flowcontrol.apiserver.k8s.io/v1	beta2
false	PriorityLevelConfiguration	ingresses	classes
networking.k8s.io/v1	false	IngressClass	ingresses
networking.k8s.io/v1	true	Ingress	networkpolicies
netpol	networking.k8s.io/v1	true	NetworkPolicy
runtimeclasses	node.k8s.io/v1	false	RuntimeClass
poddisruptionbudgets	pd	policy/v1	true
PodDisruptionBudget	podsecuritypolicies	psp	policy/v1
beta1	false	PodSecurityPolicy	clusterrolebindings
rbac.authorization.k8s.io/v1	false	ClusterRoleBinding	clusterroles
rbac.authorization.k8s.io/v1	false	ClusterRole	rolebindings
rbac.authorization.k8s.io/v1	true	RoleBinding	roles
rbac.authorization.k8s.io/v1	true	Role	priorityclasses
pcs	scheduling.k8s.io/v1	false	PriorityClass
cs	storage.k8s.io/v1	false	CSIDriver
csi	storage.k8s.io/v1	false	CSIStorageCapacity
storage.k8s.io/v1	true	CSIStorageCapacity	storageclasses
storage.k8s.io/v1	false	StorageClass	volumeattachments
storage.k8s.io/v1	false	VolumeAttachment	

Output options Use the following sections for information about how you can format or sort the output of certain commands. For details about which commands support the various output options, see the kubectl reference documentation.

Formatting output The default output format for all kubectl commands is the human readable plain-text format.

To output details to your terminal window in a specific format, you can add either the -o or --output flags to a supported kubectl command.

Syntax kubectl [command] [TYPE] [NAME] -o <output_format>

Depending on the kubectl operation, the following output formats are supported:

Output format	Description
-o custom-columns=<spec>	Print a table using a comma separated list of custom columns.
-o custom-columns-file=<filename>	Print a table using the custom columns template in the <filename> file.

jsonOutput a JSON formatted API object.-o jsonpath=<template>Print the fields defined in a jsonpath expression.-o jsonpath-file=<filename>Print the fields defined by the jsonpath expression in the <filename> file.-o kyamlOutput a KYAML formatted API object (beta).-o namePrint only the resource name and nothing else.-o wideOutput in the plain-text format with any additional information. For pods, the node name is included.-o yamlOutput a YAML formatted API object. KYAML is an experimental Kubernetes-specific dialect of YAML, and can be parsed as YAML.ExampleIn this example, the following command outputs the details for a single pod as a YAML formatted object:kubectl get pod web-pod-13je7 -o yaml

Remember: See the kubectl reference documentation

for details about which output format is supported by each command.Custom columnsTo define custom columns and output only the details that you want into a table, you can use the custom-columns option.

You can choose to define the custom columns inline or use a template file: -o custom-columns=<spec> or -o custom-columns-file=<filename>.ExamplesInline:kubectl get pods <pod-name> -o custom-columns=NAME:.metadata.name,RSRC:.metadata.resourceVersion

Template file:kubectl get pods <pod-name> -o custom-columns-file=template.txt

where the template.txt file contains:NAME RSRC

metadata.name metadata.resourceVersion

The result of running either command is similar to:NAME RSRC

submit-queue 610995

Server-side columnskubectl supports receiving specific column information from the server about objects.

This means that for any given resource, the server will return columns and rows relevant to that resource, for the client to print.

This allows for consistent human-readable output across clients used against the same cluster, by having the server encapsulate the details of printing.This feature is enabled by default. To disable it, add the

--server-print=false flag to the kubectl get command.ExamplesTo print information about the status of a pod, use a command like the following:kubectl get pods <pod-name> --server-print=false

The output is similar to:NAME AGE

pod-name 1m

Sorting list objectsTo output objects to a sorted list in your terminal window, you can add the --sort-by flag to a supported kubectl command. Sort your objects by specifying any numeric or string field

with the --sort-by flag. To specify a field, use a jsonpath expression.Syntaxkubectl [command] [TYPE] [NAME] --sort-by=<jsonpath_exp>

ExampleTo print a list of pods sorted by name, you run:kubectl get pods --sort-by=.metadata.name

Examples: Common operationsUse the following set of examples to help you familiarize yourself with running the commonly used kubectl operations:kubectl apply - Apply or Update a resource from a file or stdin.# Create a service using the definition in example-service.yaml.

kubectl apply -f example-service.yaml

Create a replication controller using the definition in example-controller.yaml.

kubectl apply -f example-controller.yaml

Create the objects that are defined in any .yaml, .yml, or .json file within the <directory> directory.

kubectl apply -f <directory>

kubectl get - List one or more resources.# List all pods in plain-text output format.

kubectl get pods

List all pods in plain-text output format and include additional information (such as node name).

kubectl get pods -o wide

List the replication controller with the specified name in plain-text output format. Tip: You can shorten and replace the 'replicationcontroller' resource type with the alias 'rc'.

kubectl get replicationcontroller <rc-name>

List all replication controllers and services together in plain-text output format.

kubectl get rc,services

List all daemon sets in plain-text output format.
kubectl get ds

List all pods running on node server01
kubectl get pods --field-selector=spec.nodeName=server01
kubectl describe - Display detailed state of one or more resources, including the uninitialized ones by default.
Display the details of the node with name <node-name>.
kubectl describe nodes <node-name>

Display the details of the pod with name <pod-name>.
kubectl describe pods/<pod-name>

Display the details of all the pods that are managed by the replication controller named <rc-name>.
Remember: Any pods that are created by the replication controller get prefixed with the name of the replication controller.
kubectl describe pods <rc-name>

Describe all pods
kubectl describe pods
Note: The kubectl get command is usually used for retrieving one or more resources of the same resource type. It features a rich set of flags that allows you to customize the output format using the -o or --output flag, for example. You can specify the -w or --watch flag to start watching updates to a particular object. The kubectl describe command is more focused on describing the many related aspects of a specified resource. It may invoke several API calls to the API server to build a view for the user. For example, the kubectl describe node command retrieves not only the information about the node, but also a summary of the pods running on it, the events generated for the node etc.
kubectl delete - Delete resources either from a file, stdin, or specifying label selectors, names, resource selectors, or resources.
Delete a pod using the type and name specified in the pod.yaml file.
kubectl delete -f pod.yaml

Delete all the pods and services that have the label '<label-key>=<label-value>'.
kubectl delete pods,services -l <label-key>=<label-value>

Delete all pods, including uninitialized ones.
kubectl delete pods --all
kubectl exec - Execute a command against a container in a pod.
Get output from running 'date' from pod <pod-name>. By default, output is from the first container.
kubectl exec <pod-name> -- date

Get output from running 'date' in container <container-name> of pod <pod-name>.
kubectl exec <pod-name> -c <container-name> -- date

Get an interactive TTY and run /bin/bash from pod <pod-name>. By default, output is from the first container.
kubectl exec -ti <pod-name> -- /bin/bash
kubectl logs - Print the logs for a container in a pod.
Return a snapshot of the logs from pod <pod-name>.
kubectl logs <pod-name>

Start streaming the logs from pod <pod-name>. This is similar to the 'tail -f' Linux command.
kubectl logs -f <pod-name>

kubectldiff - View a diff of the proposed updates to a cluster.# Diff resources included in "pod.json".
kubectldiff -f pod.json

Diff file read from stdin.

cat service.yaml | kubectldiff -f -

Examples: Creating and using pluginsUse the following set of examples to help you familiarize yourself with writing and using kubectldiff plugins:# create a simple plugin in any language and name the resulting executable file

so that it begins with the prefix "kubectldiff-"

cat ./kubectldiff-hello

#!/bin/sh

this plugin prints the words "hello world"

echo "hello world"

With a plugin written, let's make it executable:chmod a+x ./kubectldiff-hello

and move it to a location in our PATH

sudo mv ./kubectldiff-hello /usr/local/bin

sudo chown root:root /usr/local/bin

You have now created and "installed" a kubectldiff plugin.

You can begin using this plugin by invoking it from kubectldiff as if it were a regular command

kubectldiff hello

hello world

You can "uninstall" a plugin, by removing it from the folder in your

\$PATH where you placed it

sudo rm /usr/local/bin/kubectldiff-hello

In order to view all of the plugins that are available to kubectldiff, use

the kubectldiff plugin list subcommand:kubectldiff plugin list

The output is similar to:The following kubectldiff-compatible plugins are available:

/usr/local/bin/kubectldiff-hello

/usr/local/bin/kubectldiff-foo

/usr/local/bin/kubectldiff-bar

kubectldiff plugin list also warns you about plugins that are not

executable, or that are shadowed by other plugins; for example:sudo chmod -x /usr/local/bin/kubectldiff-foo #

remove execute permission

kubectldiff plugin list

The following kubectldiff-compatible plugins are available:

/usr/local/bin/kubectldiff-hello

/usr/local/bin/kubectldiff-foo

- warning: /usr/local/bin/kubectldiff-foo identified as a plugin, but it is not executable

/usr/local/bin/kubectldiff-bar

error: one plugin warning was found

You can think of plugins as a means to build more complex functionality on top of the existing kubectldiff commands:cat ./kubectldiff-whoami

The next few examples assume that you already made kubectldiff-whoami have the following contents:#!/bin/bash

```
# this plugin makes use of the `kubectl config` command in order to output
# information about the current user, based on the currently selected context
kubectl config view --template='{{ range .contexts }}{{ if eq .name "$(kubectl config current-context)" }}Current
user: {{ printf "%s\n" .context.user }}{{ end }}{{ end }}'
Running the above command gives you an output containing the user for the
current context in your KUBECONFIG file:# make the file executable
sudo chmod +x ./kubectl-whoami
```

```
# and move it into your PATH
sudo mv ./kubectl-whoami /usr/local/bin
```

kubectl whoami

Current user: plugins-user

What's nextRead the kubectl reference documentation:the kubectl command referencethe command line arguments referenceLearn about kubectl usage conventionsRead about JSONPath support in kubectlRead about how to extend kubectl with pluginsTo find out more about plugins, take a look at the example CLI plugin.FeedbackWas this page helpful?Yes

NoThanks for the feedback. If you have a specific, answerable question about how to use Kubernetes, ask it on Stack Overflow.

Open an issue in the GitHub Repository if you want to report a problem

or

suggest an improvement.Last modified March 15, 2026 at 3:21 PM PST: fix: replace deprecated argument

`--cpu-percent` with `--cpu` (af93a0a732)